Petit rapport sur la 1ére essaie d’entrainement IA

Année universitaire : 2025-2026

Objectif du projet
Développer un système de diagnostic intelligent capable de détecter automatiquement les types de pannes d'un moteur DC 12V à partir de données issues de 3 capteurs : courant, tension et vitesse. Comparer 5 à 6 algorithmes de Machine Learning et proposer deux versions complémentaires : une basée sur les labels réels du datasheet et une basée sur des formules physiques réalistes.

1. Introduction

La maintenance prédictive des équipements industriels est un enjeu majeur dans le domaine de l'ingénierie electrique et mecanique. Les moteurs a courant continu (DC) 12V sont largement utilises dans les systemes embarques, la robotique et l'automatisation industrielle. Leur defaillance peut provoquer des arrets de production couteux ou des situations dangereuses.

Ce projet de fin d'annee a pour objectif de concevoir et d'evaluer un systeme de diagnostic automatique base sur le Machine Learning, capable de classifier l'etat d'un moteur DC 12V en trois categories : fonctionnement Normal, Surcharge, et Court-Circuit.

1.1 Capteurs utilises

Capteur	Grandeur mesuree	Role dans le diagnostic
Capteur de courant	Courant I (A)	Detecte les surintensites caracteristiques de surcharges et courts-circuits
Capteur de tension	Tension V (V)	Surveille les chutes de tension symptomatiques des defauts electriques
Capteur de vitesse	Vitesse RPM	Detecte les ralentissements et anomalies mecaniques

1.2 Classes cibles

Classe	Description	Signatures physiques
Normal	Fonctionnement nominal du moteur	I ~ 10A, V ~ 220V, T ~ 50C, vibrations stables
Court-Circuit	Chemin de resistance quasi nulle	Spike de courant + chute brusque de tension
Surcharge	Charge mecanique ou thermique excessive	Courant eleve + echauffement + vibrations accrues

2. Description du Dataset

2.1 Caracteristiques generales

Propriete	Valeur
Nom du fichier	iot_equipment_monitoring_dataset.csv
Nombre d'echantillons	50 000 lignes
Nombre de colonnes	17 colonnes
Type de donnees	Synthetique (genere numeriquement)
Colonne cible originale	Fault_Type (NaN / Electrical Fault / Overheating / Mechanical Failure)
Colonne indicatrice	Fault_Status (0 = Normal, 1 = Panne)

2.2 Colonnes du dataset

Colonne	Type	Description
Timestamp	datetime	Horodatage de la mesure
Sensor_ID	string	Identifiant du capteur
Temperature	float	Temperature du moteur (C)
Vibration	float	Niveau de vibration
Pressure	float	Pression
Voltage	float	Tension d'alimentation (V)
Current	float	Courant electrique (A)
FFT_Feature1/2	float	Caracteristiques spectrales [0..1]
Normalized_*	float	Versions normalisees des capteurs (correlation = 1.0 avec originaux)
Anomaly_Score	float	Score d'anomalie [0..1]
Fault_Type	string	Type de panne (NaN si normal)
Fault_Status	int	0 = Normal, 1 = Panne

2.3 Distribution originale des labels (4 classes)

Classe originale	Nombre	Pourcentage
Normal (Fault_Status=0)	34 988	69.98%
Electrical Fault	5 034	10.07%
Overheating	5 028	10.06%
Mechanical Failure	4 950	9.90%

2.4 Probleme majeur identifie : labels independants des features

Une analyse statistique ANOVA (Analysis of Variance) a revele que les valeurs des capteurs sont statistiquement identiques entre toutes les classes de pannes. Cela signifie que les labels ont ete assignes de maniere aleatoire, independamment des mesures physiques.

Feature	F-statistique	p-value	Signal discriminant ?
Temperature	0.7795	0.5052	NON
Vibration	3.2680	0.0203	Faible
Pressure	0.9101	0.4351	NON
Voltage	0.7994	0.4940	NON
Current	0.1023	0.9587	NON
FFT_Feature1	0.6433	0.5871	NON
FFT_Feature2	2.3685	0.0686	NON
Anomaly_Score	0.0156	0.9973	NON

Implication directe

Tout modele ML entraîne sur ces labels reels obtiendra ~70% d'accuracy (baseline naif = toujours predire 'Normal') avec un Macro F1 autour de 27-31%. Ce n'est pas une limite algorithmique mais une limite fondamentale du dataset. Deux notebooks finaux ont ete produits pour documenter cette realite et proposer une alternative physique.

3. Evolution du Projet — Historique des Versions

Le projet a evolue a travers plusieurs versions successives. Chaque version a apporte des corrections et des ameliorations basees sur l'analyse des resultats precedents.

3.1 Version initiale — Script Python simple

Point de depart : un script Python (.py) d'entrainement de modeles RF et MLP fourni par l'equipe. Fonctionnalites initiales :

- Lecture du CSV et detection automatique des colonnes capteurs via regex
- Extraction de features depuis des fenetres temporelles (si > 5 colonnes du meme type)
- Entrainement Random Forest (n=200) et MLP sklearn
- Sauvegarde des modeles avec joblib

3.2 v1 — Conversion en notebook Colab

- Transformation du script en fichier .ipynb structure en 11 sections
- Ajout de visualisations (pie chart, bar chart, confusion matrix, feature importance)
- Ajout de early_stopping=True sur le MLP
- Fonction predict_fault() pour la demonstration

3.3 v2 — Adaptation pour VS Code

- Suppression de tout le code specifique a Colab (drive.mount, files.upload/download)
- Cellule d'installation automatique via subprocess
- Chemins locaux avec os.path.join() — compatible Windows/Linux/Mac
- Creation automatique du dossier outputs/

3.4 Corrections de bugs (entre v2 et v3)

Bug	Cause	Correction
TypeError sur classification_report	le.classes_ contenant des float	CLASS_NAMES = [str(c) for c in le.classes_]
NaN/Inf dans la matrice X	Division par zero dans l'extraction de features	np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)
UndefinedMetricWarning	Classes absentes du jeu de test	zero_division=0 dans toutes les metriques
loss_curve_ absente	Early stopping avant la 1ere iteration	Guard hasattr(mlp, 'loss_curve_')
predict_fault() mal callee	Valeurs capteurs aux mauvaises positions	Recherche par regex des colonnes correspondantes

3.5 v3 — Adaptation au vrai dataset + label NaN corriges

Apres inspection du CSV reel, plusieurs problemes ont ete identifies et corriges :

- Fault_Status == 0 genere Fault_Type = NaN : correction par label construction

```
df["Label"] = df.apply(lambda r: "Normal" if r["Fault_Status"]==0 else
str(r["Fault_Type"]), axis=1)
```

- 4 classes finales propres : Normal, Electrical Fault, Overheating, Mechanical Failure
- 13 features exactes du CSV utilisees directement
- Ajout boxplots par classe et matrice de correlation
- predict_fault() avec les 13 parametres reels

3.6 v4 — Tentative d'amelioration (oversampling + feature engineering)

Tentative d'amelioration des metriques avec :

- 6 nouvelles features engineerees (ratios, produits physiques)
- Oversampling aleatoire pour rééquilibrer les classes
- Ajout de HistGradientBoosting comme 3eme modele
- Ensemble soft voting

Resultats catastrophiques :

Modele	Accuracy v4
Random Forest	69.97%
HistGradBoosting	38.38%
MLP	Timeout (ne terminait pas)

3.7 Diagnostic des causes d'echec (v4)

Probleme	Cause racine	Solution v5+
HGB a 38%	Oversampling duplique 40k -> 112k echantillons. HGB tres sensible aux doublons	sample_weight puis class_weight='balanced'
MLP timeout	112k echantillons apres oversampling = entrainement impossible	Undersample equilibre (5k/classe max)
Features redondantes	Normalized_* ont correlation=1.0 avec les originaux	Suppression des 5 colonnes normalisees
Oversampling inutile	Les features n'ont aucun signal : dupliquer n'apporte rien	class_weight gere le desequilibre en interne

3.8 v5 — Diagnostic complet + approche binaire

- Section de diagnostic statistique (ANOVA) documentee dans le notebook
- Preuve que 7/8 features ont $p > 0.05$ (aucun signal discriminant)
- Classification 4 classes conservee pour la methodologie
- Ajout d'une classification binaire Normal vs Panne
- Fix HGB : class_weight='balanced' au lieu de sample_weight
- Undersample equilibre pour MLP (5k/classe)

3.9 v6 — 5 Algorithmes + Ensemble (4 classes)

- Ajout de KNN (k=5, weights='distance') sur sous-ensemble equilibre
- Ajout de SVM (RBF, C=10) sur sous-ensemble equilibre (3k/classe)
- Ensemble soft voting de 5 modeles
- Macro F1 comme metrique principale (traite toutes les classes equitablement)
- Heatmap F1 par classe par algorithme

4. Notebooks Finaux — Architecture et Contenu

4.1 v6b — Labels Reels (3 classes)

Ce notebook utilise les labels originaux du dataset, mappes vers 3 classes physiquement justifiees.

4.1.1 Mapping des classes

Label original CSV	Classe finale	Justification
Fault_Status == 0 (NaN)	Normal	Fonctionnement dans les plages nominales
Electrical Fault	Court-Circuit	Defaut electrique = signature de court-circuit
Overheating	Surcharge	Surchauffe = symptome de surcharge energetique
Mechanical Failure	Surcharge	Defaillance mecanique -> surcharge du courant moteur

4.1.2 Distribution apres mapping

Classe	Nombre	Pourcentage
Normal	34 988	69.98%
Surcharge (Overheating + Mechanical)	9 978	19.96%
Court-Circuit (Electrical Fault)	5 034	10.07%

4.1.3 Resultats attendus

Algorithme	Accuracy	Macro F1	Observation
Random Forest	~70%	~27%	Predit tout 'Normal' - baseline dataset
HistGradBoosting	~32%	~28%	Tente les 3 classes mais signal faible
MLP	~41%	~31%	Meilleur Macro F1 grace a l'undersample
KNN (k=5)	~35%	~29%	Predit les 3 classes mais bruit important
SVM (RBF)	~33%	~28%	Entraine sur sous-ensemble equilibre
Ensemble (5)	variable	meilleur	Combine les forces de chaque modele

4.2 v7 — Labels Physiques (3 classes)

Ce notebook genere des labels physiquement coherents a partir de formules mathematiques appliquees aux valeurs reelles des capteurs. Cette approche produit des classes discriminantes et des performances elevees.

4.2.1 Formules de classification physique

Valeurs nominales du moteur DC 12V utilisees comme reference :

Grandeur	Valeur nominale	Ecart-type
Courant I	10.0 A	3.0 A
Tension V	220.0 V	10.0 V

Grandeur	Valeur nominale	Ecart-type
Temperature T	50.0 C	15.0 C
Vibration Vib	100.0 u.a.	25.0 u.a.

Z-scores (ecarts normalises) :

```
z_I   = (Current     - 10.0) / 3.0
z_V   = (Voltage     - 220.0) / 10.0
z_T   = (Temperature - 50.0) / 15.0
z_Vib = (Vibration   - 100.0) / 25.0
```

Score Court-Circuit (spike courant + chute tension) :

```
cc_score = z_I - z_V
Court-Circuit si cc_score > 1.5
```

Justification : un court-circuit cree un chemin de resistance quasi nulle, generant un spike de courant (z_I eleve) et une chute de tension simultanee (z_V negatif). La difference z_I - z_V capture precisement cette signature.

Score Surcharge (stress electro-thermique + mecanique) :

```
ol_score = 0.45 * z_I + 0.35 * z_T + 0.20 * z_Vib
Surcharge si ol_score > 0.6
```

Justification : une surcharge se manifeste par une augmentation du courant (poids 0.45 = contribution principale), un echauffement progressif (poids 0.35) et une augmentation des vibrations (poids 0.20 = contribution mecanique secondaire).

4.2.2 Distribution des labels physiques

Classe	Nombre	Pourcentage	Seuil
Normal	37 840	75.68%	Aucune condition verifiee
Court-Circuit	7 199	14.40%	cc_score > 1.5
Surcharge	4 961	9.92%	ol_score > 0.6 (et cc_score <= 1.5)

4.2.3 Resultats (6 algorithmes + XGBoost + Ensemble)

Algorithme	Accuracy	Macro F1	Temps
Random Forest	97.84%	95.80%	~25s
HistGradBoosting	98.46%	97.22%	~8s
MLP (128->64->32)	98.42%	97.22%	~5s
KNN (k=5)	86.30%	79.91%	~1s
SVM (RBF, C=10)	96.62%	94.39%	~2s
XGBoost	~97-98%	~95-97%	~3s
Ensemble (6 modeles)	98.41%	97.23%	-

5. Description des Algorithmes

5.1 Random Forest

Ensemble de N arbres de decision entraines sur des sous-echantillons aleatoires du dataset. Chaque arbre vote pour une classe et la majorite l'emporte.

Parametre	Valeur	Justification
n_estimators	300	Compromis precision/temps
max_features	'sqrt'	Reduit la correlation entre arbres
class_weight	'balanced'	Compense le desequilibre des classes
random_state	42	Reproductibilite des resultats

5.2 HistGradientBoosting

Boosting par gradient utilisant des histogrammes de features. Plus rapide que GradientBoosting classique et nativement compatible avec class_weight.

Parametre	Valeur	Justification
max_iter	200	Iterations de boosting
max_depth	6	Profondeur max des arbres
learning_rate	0.1	Taux d'apprentissage
class_weight	'balanced'	Fix du bug des versions precedentes (sample_weight causait 22%)

5.3 MLP — Reseau de Neurones

Perceptron multi-couches avec activation ReLU et optimiseur Adam. Entraine sur un sous-ensemble equilibre pour eviter le biais vers la classe majoritaire.

Parametre	Valeur	Justification
hidden_layer_sizes	(128, 64, 32)	Architecture progressive decroissante
solver	'adam'	Optimiseur adaptatif efficace
early_stopping	True	Arret si validation ne s'ameliore plus
Donnees	Undersample 5k/classe	Evite le timeout et le biais vers Normal

5.4 KNN — K Plus Proches Voisins

Pour chaque echantillon de test, identifie ses k voisins les plus proches dans le train et vote par majorite ponderee.

Parametre	Valeur	Justification
n_neighbors	5	Assez petit pour capter les classes minoritaires (k>20 = tout Normal)
weights	'distance'	Voisins proches ont plus de poids

Parametre	Valeur	Justification
Donnees	Undersample 5k/classe	Seul moyen de gerer le desequilibre (pas de class_weight)

5.5 SVM — Support Vector Machine

Cherche l'hyperplan de separabilite maximale entre les classes dans un espace transforme par le noyau RBF (Radial Basis Function).

Parametre	Valeur	Justification
kernel	'rbf'	Noyau radial pour frontieres non-lineaires
C	10.0	Regularisation moderee
class_weight	'balanced'	Compense le desequilibre
Donnees	Undersample 3k/classe	SVM = O(n^2) memoire. 40k -> timeout, 3k/classe -> 2s

5.6 XGBoost (v7 uniquement)

eXtreme Gradient Boosting : boosting par gradient optimise avec regularisation L1+L2 et subsampling. Tres performant sur des donnees tabulaires.

Parametre	Valeur	Justification
n_estimators	200	Iterations de boosting
max_depth	6	Profondeur par arbre
learning_rate	0.1	Taux d'apprentissage
subsample	0.8	80% des lignes par arbre (evite overfitting)
colsample_bytree	0.8	80% des features par arbre
sample_weight	Calcule	n_total / (n_classes * n_class_i) par echantillon

5.7 Ensemble Soft Voting

Combine les probabilites de tous les modeles par moyenne simple. La prediction finale est la classe ayant la probabilite moyenne la plus elevee.

```
P_ensemble = (P_RF + P_HGB + P_MLP + P_KNN + P_SVM [+ P_XGB]) / N_modeles
Prediction = argmax(P_ensemble)
```

Avantage : exploite les forces complementaires de chaque modele. Un modele confident sur une classe compense un autre moins certain.

6. Methodologie et Choix Techniques

6.1 Pipeline de traitement

Etape	Outil	Justification
Chargement	pandas.read_csv()	Lecture efficace du CSV
Construction labels	df.apply(lambda)	Mapping flexible et transparent
Selection features	Exclusion Normalized_*	Correlation=1.0 avec originaux -> redondance pure
Split train/test	train_test_split(stratify=y)	Proportions de classes identiques train et test
Normalisation	StandardScaler	Moyenne=0, ecart-type=1 pour tous les modeles
Undersample	balanced_subsample()	MLP, KNN, SVM uniquement (pas de class_weight)
Evaluation	Accuracy + Macro F1	Macro F1 traite toutes les classes equitablement
Sauvegarde	joblib.dump()	Modeles persistants pour deploiement

6.2 Gestion du desequilibre des classes

Le dataset est fortement desequilibre (~70% Normal). Chaque algorithme necessite une strategie differente :

Algorithme	Methode	Implementation
RF, HGB, SVM	class_weight='balanced'	Poids inversement proportionnels a la frequence de chaque classe
XGBoost	sample_weight	Calcul : n_total / (n_classes * count_classe)
MLP, KNN	Undersample equilibre	balanced_subsample(X, y, max_per_class=5000)

6.3 Metriques d'evaluation

Metrique	Formule	Quand l'utiliser
Accuracy	n_correct / n_total	Classes equilibrees - trompeuse ici (70% = baseline)
Macro F1	moyenne(F1 par classe)	Classes desequilibrees - metrique principale ici
Weighted F1	F1 pondere par support	Compromis entre les deux
Macro Precision	moyenne(P par classe)	Qualite des predictions positives
Macro Recall	moyenne(R par classe)	Taux de detection par classe

7. Analyse Comparative des Resultats

7.1 Comparaison v6b (labels reels) vs v7 (labels physiques)

Critere	v6b — Labels Reels	v7 — Labels Physiques
Source des labels	Fault_Status + Fault_Type du CSV	Formules z_I - z_V et 0.45*z_I + 0.35*z_T + 0.20*z_Vib
RF Accuracy	~70%	~98%
RF Macro F1	~27%	~96%
HGB Accuracy	~32%	~98%
MLP Accuracy	~41%	~98%
KNN Accuracy	~35%	~86%
Validite scientifique	Labels reels mais non-discriminants	Labels coherents mais circularite partielle
Usage recommande	Documentation honnete des limites du dataset	Demonstration de la methodologie ML

7.2 Pourquoi la v7 atteint 97-98% ?

Les modeles apprennent la fonction mathematique qui a ete utilisee pour generer les labels. C'est un phenomene de circularite : si on definit 'Court-Circuit' comme z_I - z_V > 1.5, un Random Forest avec acces aux features I et V va naturellement apprendre cette frontiere de decision. Les 2-3% d'erreur residuels correspondent aux echantillons aux frontieres des seuils.

7.3 Performances KNN vs autres algorithmes

KNN obtient des resultats inferieurs a tous les autres algorithmes dans les deux versions. Cette limitation s'explique par la 'malediction de la dimensionnalite' : avec 8 features, la notion de 'voisin proche' perd de sa signification. De plus, KNN ne supporte pas class_weight et depend entierement de l'undersample pour gerer le desequilibre.

7.4 Observations sur SVM

Le SVM obtient de bonnes performances en v7 (~96.6%) malgre un sous-ensemble reduit (3k/classe). La frontiere de decision RBF est efficace pour separer des classes definies par des seuils sur des combinaisons lineaires de features. En v6b, les resultats sont similaires aux autres algorithmes (~33%) : le signal absent dans les donnees limite tous les algorithmes de la meme facon.

8. Structure des Notebooks Finaux

8.1 v6b — dc_motor_fault_diagnosis_v6b.ipynb (45 cellules)

Section	Contenu
1. Installation	pip install automatique des librairies
2. Imports	sklearn, matplotlib, seaborn, scipy
3. Configuration	Chemin CSV, dossier outputs, tailles sous-ensembles
4. Chargement + Mapping	Labels reels -> 3 classes avec justification physique
5. Diagnostic ANOVA	Preuve statistique de l'absence de signal dans les features
6. Features + Encodage	8 features, LabelEncoder, CLASS_NAMES en strings
7. Split + Normalisation	80/20 stratifie, StandardScaler, balanced_subsample()
8-12. Entrainement	RF, HGB, MLP, KNN, SVM avec parametres documentes
13. Ensemble	Soft voting 5 modeles
14. Confusion matrices	6 matrices en 2x3
15. Comparaison	Tableau + graphique Accuracy vs Macro F1 + heatmap F1
16. Feature importance	RF Gini importance
17. Sauvegarde	7 fichiers .joblib dans outputs_v6b/
18. Prediction	predict_motor() avec 8 parametres capteurs

8.2 v7 — dc_motor_fault_diagnosis_v7.ipynb (48 cellules)

Section	Contenu
1-3. Setup	Installation, imports, configuration
4. Chargement	CSV brut
5. Labels physiques	Formules z-scores + seuils + assignation + assertion
6. Visualisation scores	Histogrammes des scores CC et OL par classe + scatter 2D
7. Features + Encodage	8 features originales, LabelEncoder
8. Split + Sous-ensembles	80/20, StandardScaler, 3 sous-ensembles equilibres
9-14. Entrainement	RF, HGB, MLP (+courbe loss), KNN, SVM, XGBoost
15. Ensemble	Soft voting 6 modeles
16. Confusion matrices	7 matrices en 3x3
17. Comparaison	Tableau, graphiques, heatmap F1
18. Feature importance	RF + XGBoost cote a cote
19. Sauvegarde	8 fichiers .joblib dans outputs/
20. Prediction	predict_motor() avec verification par les formules physiques

9. Conclusions et Perspectives

9.1 Bilan du projet

Ce projet a abouti a la creation de deux notebooks complets et complementaires pour le diagnostic de moteurs DC 12V par Machine Learning. Cinq algorithmes ont ete implementes, compares et documentes de maniere rigoureuse.

Points cles accomplis :

- Conversion d'un script Python en notebook structure et documenté pour VS Code
- Identification et correction de multiples bugs (TypeError, NaN, oversampling excessif)
- Diagnostic statistique prouvant les limites du dataset synthetique
- Deux approches complementaires (labels reels vs labels physiques)
- 6 algorithmes implementes et compares : RF, HGB, MLP, KNN, SVM, XGBoost
- Ensemble soft voting combinant tous les modeles
- Visualisations completes : confusion matrices, heatmaps F1, feature importance

9.2 Lecons apprises

Lecon	Detail
Qualite des donnees > algorithme	Avec un dataset synthetique non-discriminant, aucun algorithme ne peut depasser 70%
class_weight vs oversampling	class_weight est toujours preferable pour les arbres et SVM. L'oversampling cause des problemes de performance et de qualite
Macro F1 > Accuracy	Sur des donnees desequilibrees, l'accuracy peut etre trompeuse. Macro F1 est la metrique honnete
KNN sensible a la dimension	Avec 8 features, KNN est desavantage par rapport aux modeles basés sur les arbres
SVM limite par la taille	Le noyau RBF est O(n^2) : necessaire de reduire le dataset pour des temps raisonnables
Circularite des labels physiques	Generer des labels depuis les features puis entrainer un modele sur ces memes features produit artificiellement de bonnes performances

9.3 Recommandations pour un vrai projet industriel

- Utiliser un dataset avec un signal reel entre les capteurs et les pannes (ex: NASA CMAPS, CWRU Bearing Dataset)
- Privilegier HistGradientBoosting ou XGBoost pour les donnees tabulaires desequilibrees
- Implementer une detection d'anomalie (Isolation Forest, Autoencoder) en complement de la classification
- Collecter des donnees reelles sur le moteur DC cible avec etiquetage manuel par un expert
- Envisager des features temporelles (spectres FFT complets, tendances sur fenetre glissante)

9.4 Perspectives d'amelioration

- Hyperparameter tuning (GridSearchCV ou Optuna) sur les modeles les plus prometteurs
- Test de SMOTE (Synthetic Minority Oversampling) comme alternative a l'undersample pour MLP
- Integration d'un module de monitoring en temps reel avec les capteurs physiques
- Deploiement sur microcontroleur (ESP32, Arduino) avec modele compresse (ONNX)

Annexes

A. Liste des fichiers produits

Fichier	Description	Version
dc_motor_fault_diagnosis_v6b.ipynb	Notebook labels reels, 3 classes, 5 algorithmes	Finale
dc_motor_fault_diagnosis_v7.ipynb	Notebook labels physiques, 3 classes, 6 algos + XGBoost	Finale
outputs_v6b/rf_model.joblib	Modele RF entraine sur labels reels	v6b
outputs_v6b/hgb_model.joblib	Modele HGB entraine sur labels reels	v6b
outputs_v6b/mlp_model.joblib	Modele MLP entraine sur labels reels	v6b
outputs_v6b/knn_model.joblib	Modele KNN entraine sur labels reels	v6b
outputs_v6b/svm_model.joblib	Modele SVM entraine sur labels reels	v6b
outputs/rf_model.joblib	Modele RF entraine sur labels physiques	v7
outputs/hgb_model.joblib	Modele HGB entraine sur labels physiques	v7
outputs/mlp_model.joblib	Modele MLP entraine sur labels physiques	v7
outputs/knn_model.joblib	Modele KNN entraine sur labels physiques	v7
outputs/svm_model.joblib	Modele SVM entraine sur labels physiques	v7
outputs/xgb_model.joblib	Modele XGBoost entraine sur labels physiques	v7
outputs/scaler.joblib	StandardScaler ajuste sur le train	v7
outputs/label_encoder.joblib	LabelEncoder pour decoder les predictions	v7
outputs/feature_cols.joblib	Liste des 8 features utilisees	v7

B. Librairies Python utilisees

Librairie	Version	Usage
scikit-learn	1.8.0	RF, HGB, MLP, KNN, SVM, metriques, preprocessing
xgboost	latest	XGBoostClassifier
pandas	3.0.1	Lecture CSV, manipulation DataFrame
numpy	latest	Calcul numerique, matrices
matplotlib	latest	Visualisations, graphiques
seaborn	latest	Heatmaps, statistiques visuelles

Librairie	Version	Usage
scipy	latest	Test ANOVA (f_oneway)
joblib	latest	Sauvegarde et chargement des modeles

C. Glossaire

Terme	Definition
Accuracy	Proportion d'echantillons correctement classes parmi tous les echantillons
Macro F1	Moyenne des F1-scores individuels de chaque classe, sans ponderation par la taille
ANOVA	Analysis of Variance - test statistique pour comparer les moyennes de plusieurs groupes
class_weight='balanced'	Poids wi = n_total / (n_classes * n_i) pour chaque classe i
Undersample	Reduction du nombre d'echantillons des classes pour les equilibrer
Soft Voting	Combinaison des probabilites (et non des votes) de plusieurs modeles
Z-score	Nombre d'ecarts-types d'une valeur par rapport a la moyenne : z = (x - mu) / sigma
Baseline naif	Performance minimale obtenue en predisant toujours la classe majoritaire
Macro F1 vs Weighted F1	Macro: donne meme poids a chaque classe. Weighted: ponderee par le nombre d'echantillons